

§11.4.141 Token-Efficiency Baseline — Always-Loaded Governance Cost

Revision: 1 **Last modified:** 2026-06-09T00:00:00Z **Description:** REAL measured baseline of the always-loaded governance-context token cost (the static block that loads every conversation turn), captured as §11.4.141 evidence. Verifies/corrects the design's “~170K tokens/turn” claim with a measured number, and computes the projected prompt-caching savings at Opus 4.8 input pricing. **Authority:** §11.4.141 (token-efficiency) — anti-bluff §11.4.6 (real measured numbers, exact method stated; no guessing). **Scope:** Project layer (ATMOSphere-Android-15 consumer governance + constitution submodule governance).

1. Measurement method (stated exactly, per §11.4.6)

Tokenizer used: tiktoken cl100k_base encoding, invoked via python3. tiktoken was **not preinstalled** in this environment; it was installed at measurement time via pip3 install tiktoken. import tiktoken; enc = tiktoken.get_encoding("cl100k_base"), then len(enc.encode(open(f, encoding="utf-8").read())) per file. Byte counts via wc -c, char counts via Python len(str).

Why cl100k and not the Claude tokenizer: Anthropic's exact production tokenizer is not available as an offline local library. The authoritative on-Claude method is the POST /v1/messages/count_tokens endpoint (Anthropic SDK client.messages.count_tokens), which requires a live API call + key and was **not** available in this measurement environment. cl100k_base (OpenAI's GPT-3.5/4 tokenizer) is the best defensible **offline** approximation; it is a different tokenizer and **undercounts** Claude tokens on English-prose + Markdown by a documented ~10-20% (and more on code). This report therefore reports BOTH the raw cl100k measurement (a hard floor) AND a Claude-tokenizer estimate **band** of $\text{cl100k} \times 1.10 \dots \times 1.20$.

Uncertainty statement (anti-bluff): the cl100k numbers are EXACT measurements of the cl100k tokenization. The Claude-tokenizer numbers are an **estimate band**, not a measurement —

to convert the band into a measurement, run
client.messages.count_tokens(model="claude-opus-4-8",
messages=[{"role":"user","content": <file contents>}]) per file
with a live key and replace the band with the returned
input_tokens. That follow-up is the one remaining
PENDING_FORENSICS: item for an exact Claude-tokenizer total.

2. Per-file token counts (cl100k_base, measured)

Group	File	Bytes	Chars	Tokens (cl100k)	bytes/tok
consumer	CLAUDE.md	361,977	358,202	95,894	3.77
consumer	AGENTS.md	212,549	209,817	56,445	3.77
consumer	QWEN.md	84,467	83,283	23,039	3.67
constitution	constitution/ Constitution.md	781,977	773,118	192,794	4.06
constitution	constitution/ CLAUDE.md	318,509	314,009	83,462	3.82
constitution	constitution/ AGENTS.md	244,010	240,661	65,575	3.72
constitution	constitution/ QWEN.md	145,601	143,469	38,556	3.78
TOTAL (all 7)		2,149,090	2,122,559	555,765	3.87

Sub-totals:

- consumer trio (CLAUDE.md + AGENTS.md + QWEN.md) = **175,378** tokens
- constitution quad (Constitution.md + CLAUDE.md + AGENTS.md + QWEN.md) = **380,387** tokens

cl100k cross-check against the chars/divisor heuristic on the 7-file total (2,122,559 chars): /3.5 → 606,445; /4.0 → 530,640. The measured cl100k total (555,765) sits between these, as expected.

3. What ACTUALLY loads every turn (the load-bearing distinction)

The 7-file 555,765-token figure is the full governance corpus. **It is NOT what Claude Code injects each turn.** Two facts narrow it down:

- 1. **AGENTS.md and QWEN.md are NOT auto-loaded by Claude Code.** They are the sibling agent-instruction files for other CLIs (Codex/Aider/Cursor read AGENTS.md; Qwen Code reads QWEN.md). In a Claude Code session only CLAUDE.md (+ its @imports) is auto-injected.
- 2. **The consumer CLAUDE.md @imports constitution/CLAUDE.md** (line 19: @constitution/CLAUDE.md). Claude Code resolves @imports recursively, so the constitution CLAUDE.md content loads **transitively** with the consumer CLAUDE.md.

Therefore the **always-loaded-every-turn static governance block** in a Claude Code session is:

Component	Tokens (cl100k)
consumer CLAUDE.md	95,894
@imported constitution/ CLAUDE.md	83,462
Always-loaded total (cl100k, measured)	179,356
Always-loaded total (Claude-tokenizer estimate, ×1.10-1.20)	~197,000 - ~215,000

(Confirmed live: the system-reminder at the start of THIS conversation injected exactly the consumer CLAUDE.md + constitution/CLAUDE.md pair — matching the auto-load model above.)

4. Confirm / correct the design’s “~170K tokens/turn” claim

Verdict: the ~170K estimate is APPROXIMATELY CORRECT at the cl100k floor, and is a slight UNDER-count of the Claude-tokenizer reality.

- The **measured always-loaded block** is **179,356 cl100k tokens** — $179,356 / 170,000 = 1.032$, i.e. ~3% above the 170K estimate at the cl100k floor.

- The design figure also lines up almost exactly with the **consumer trio** (175,378 cl100k) — within ~3% — which is consistent with “~170K” having been an estimate of the consumer-side governance (CLAUDE+AGENTS+QWEN) rather than of the strict Claude-Code-auto-loaded CLAUDE.md+@import pair. Both interpretations land ~170-180K cl100k.
- On the **actual Claude tokenizer**, the always-loaded block is estimated at **~197K - ~215K tokens** (cl100k undercounts). So the true per-turn governance load is **higher** than 170K, by roughly 15-25%.

Correction: use **~179K cl100k (measured)** / **~197K-215K Claude-est** as the baseline, not a flat 170K. 170K is a fair order-of-magnitude estimate but reads ~15-25% low against the real (Claude-tokenizer) cost.

5. Cost per turn + projected cached savings (Opus 4.8 pricing)

Pricing (Opus 4.8, claude-opus-4-8, from the bundled claude-api skill, cached 2026-05-26): input **\$5.00 / 1M tokens**; cache read $\approx 0.1\times = \$0.50 / 1M$; cache write $1.25\times$ (5-min TTL) / $2.0\times$ (1-hour TTL).

Cost of the **static governance prefix alone**, per turn:

Scenario	Tokens	\$/turn (prefix)
Full price (no cache), cl100k	179,356	\$0.8968
Full price (no cache), Claude-est	197,000-215,000	\$0.99 - \$1.08
Cached read (0.1 $\times$), cl100k	179,356	\$0.0897
Cached read (0.1 $\times$), Claude-est	197,000-215,000	\$0.099 - \$0.108

Projected savings from caching this static block (per turn):

- cl100k: **\$0.8968 $\rightarrow$ \$0.0897 = \$0.807 saved/turn (90% off the prefix)**
- Claude-est: **\$0.89 - \$0.97 saved/turn**

One-time cache-write surcharge (paid once per TTL window when the prefix is first written):

- 5-min TTL (+0.25×): cl100k **\$0.224** (Claude-est ~\$0.25-\$0.27)
- 1-hour TTL (+1.0×): cl100k **\$0.897** (Claude-est ~\$0.99-\$1.08)

Break-even: with the 5-min TTL the +0.25× write surcharge is recovered after the **first** cached read (each cached turn saves $0.9\times \gg$ the one-time $0.25\times$ write), so caching is net-positive from the 2nd turn onward in any multi-turn session. The 1-hour TTL doubles the write cost (+1.0×) and needs ≥ 2 reads within the hour to pay off — preferable only for bursty sessions with idle gaps > 5 min.

6. Exact Claude Code `cache_control` enablement for this static block

The governance block is the ideal cache target because it is **byte-stable across turns** and sits at the **front of the prefix** (render order is tools → system → messages; CLAUDE.md content is injected into the system/early-context region ahead of the volatile per-turn user messages). Caching is a **prefix match** — anything stable-and-early caches cleanly; the volatile per-turn content (the user's new message, timestamps, tool results) must stay **after** the last `cache_control` breakpoint.

Where the breakpoint goes (API shape): a single `cache_control: {"type": "ephemeral"}` on the **last block of the static governance prefix** (i.e. the end of the injected CLAUDE.md/system region) caches the whole governance block in one breakpoint. In raw Messages-API terms:

```
"system": [  
  { "type": "text",  
    "text": "<consumer CLAUDE.md + @imported constitution/  
CLAUDE.md – the always-loaded block>",  
    "cache_control": { "type": "ephemeral" } }      // 5-min TTL  
  (default)  
]  
// For bursty sessions with >5-min gaps, use 1h TTL on the SAME  
block:  
//   "cache_control": { "type": "ephemeral", "ttl": "1h" }
```

Constraints that apply to this block (from the `claude-api/prompt-caching` reference):

- **Max 4 `cache_control` breakpoints per request;** one suffices for the governance prefix.

- **Minimum cacheable prefix on Opus 4.8 = 4,096 tokens.**
The 179K-token block is far above the floor, so it always caches (no silent miss).
- **Verify hits** via `response.usage.cache_read_input_tokens` (should be ~179K cl100k after the first turn) and `cache_creation_input_tokens` (non-zero only on the first/refresh turn). If `cache_read_input_tokens` is 0 across turns, a silent invalidator changed the prefix bytes (e.g. an interpolated timestamp/date inside the governance block, a non-deterministic ordering, or an @import target edit). The \$12.10/\$11.44 “Last modified” timestamps live INSIDE these governance files — **any edit to CLAUDE.md or constitution/CLAUDE.md invalidates the cache** and forces one full-price re-write next turn (then re-caches).

In the Claude Code harness specifically: the harness owns the request assembly and sets `cache_control` on the static system/CLAUDE.md prefix automatically (prompt caching for the system + CLAUDE.md block is on by default for Claude Code sessions on caching-capable models like Opus 4.8). There is no per-project `settings.json` switch the operator must flip to cache THIS block — it caches because it is the stable leading prefix. The operator’s lever is keeping the governance files **byte-stable within a session** (don’t edit CLAUDE.md mid-session) so the cached prefix survives; and, for editing sessions, accepting that the first turn after any governance edit pays the ~\$0.90 (cl100k) / ~\$1.00–\$1.08 (Claude-est) full-price re-write once, amortized to ~\$0.09–\$0.11/turn thereafter.

7. One-line summary

Measured always-loaded governance = **179,356 cl100k tokens/turn** (method: tiktoken cl100k_base, offline) → **~197K-215K on the Claude tokenizer** (cl100k undercounts ~10-20%; exact figure PENDING_FORENSICS: via `count_tokens`). This **confirms the ~170K design estimate at the cl100k floor (within ~3%) and corrects it ~15-25% upward** for the real tokenizer. At Opus 4.8 pricing the static prefix costs **\$0.90/turn full price → \$0.09/turn cached (≈90% / ~\$0.81/turn saved, cl100k)**, net-positive from the 2nd turn under the default 5-min ephemeral cache on the leading CLAUDE.md/system block.